Sesión 9 · Módulo 3: Modelos de clasificación

Teorema de Bayes · Gini vs ganancia de información · Curva ROC y AUC

Reseñas de cine/cocina · Diagnóstico de diabetes · scikit-learn

Plan de la sesión (≈45 min)

Qué veremos

- **A.** Teorema de Bayes y los clasificadores Naive Bayes (gaussiano, multinomial, Bernoulli)
- **B.** Árboles de decisión para clasificación: impureza de Gini vs ganancia de información
- **C.** De la matriz de confusión a la curva ROC y el área bajo la curva (AUC)

Dos familias de modelos muy distintas (una "cuenta probabilidades", la otra "hace preguntas de sí o no") y una forma más completa de evaluar cualquier clasificador.

De dónde venimos: seguimos en el Módulo 3

Módulo 3 (clasificación): lo visto hasta ahora

Sesión	Tema
8	Regresión logística, kNN, exactitud/precisión/recall/F1
9 (hoy)	Naive Bayes, árboles de clasificación, ROC/AUC

Lo que se mantiene

Seguimos prediciendo una **categoría**, y seguimos usando `Pipeline` y validación cruzada. La matriz de confusión de la sesión pasada es la base de **todo** lo que viene hoy.

Siguiente sesión del módulo: máquinas de vectores de soporte (SVM) para clasificación.

A1 · Repaso: el teorema de Bayes

La fórmula

$$P(y \mid x) = \frac{P(x \mid y)\, P(y)}{P(x)}$$

- $P(y)$: **prior**, qué tan común es la clase y antes de ver los datos.
- $P(x \mid y)$: **verosimilitud**, qué tan típicos son los datos x si la clase fuera y.
- $P(y \mid x)$: **posterior**, lo que realmente queremos: la clase dado lo observado.

Un clásico: la prueba médica

Prevalencia 8 %, sensibilidad 95 %, especificidad 90 %. Si la prueba sale **positiva**:

$$P(D \mid +) = \frac{0.95 \times 0.08}{0.95 \times 0.08 + 0.10 \times 0.92} \approx 0.45$$

Solo 45 %, aunque la prueba "parezca" muy buena: la prevalencia **importa**.

A2 · De Bayes a un clasificador

La idea

Con varias clases posibles, se elige la de **mayor probabilidad posterior**:

$$\hat{y} = \arg\max_{y} \; P(y \mid x) = \arg\max_{y} \; P(x \mid y)\, P(y)$$

($P(x)$ no depende de y, así que no afecta la decisión y se puede ignorar).

El problema

Si $x = (x_1, \ldots, x_p)$ tiene muchas variables, estimar $P(x \mid y)$ **conjunta** requiere combinaciones imposibles de observar con datos reales.

A3 · El supuesto “naive”: independencia condicional

La simplificación

Se asume que, **dada la clase**, las variables son independientes entre sí:

$$P(x_1, \ldots, x_p \mid y) \approx \prod_{i=1}^{p} P(x_i \mid y)$$

Así, cada variable se estima **por separado**: mucho más simple y requiere pocos datos.

¿Y si es falso?

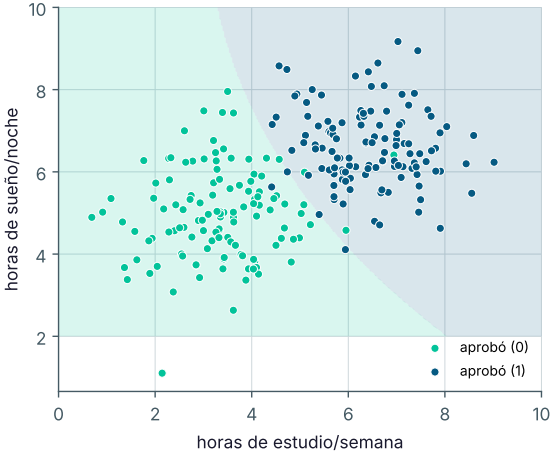
Casi nunca las variables son realmente independientes (p. ej. glucosa e IMC están correlacionados). Aun así, Naive Bayes suele funcionar **bien en la práctica**: para clasificar solo importa el orden relativo de las probabilidades, no su valor exacto.

A4 · Tres versiones, según el tipo de variable

Variante	Supone que $x_i \mid y$ sigue...	Ejemplo típico
Gaussiano	una distribución normal	variables continuas (glucosa, edad)
Multinomial	una distribución de conteos	frecuencia de palabras en un texto
Bernoulli	una distribución 0/1	presencia/ausencia de una palabra

Las tres comparten la misma idea (Bayes + independencia); solo cambia **cómo** se estima $P(x_i \mid y)$ para cada variable.

A5 · Naive Bayes gaussiano, en datos



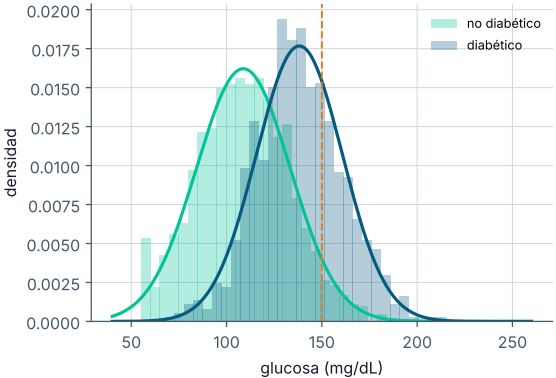
Cómo decide

Para cada clase y cada variable, se ajusta una **normal** (media y varianza); la frontera de decisión sale de comparar las densidades ponderadas por el prior.

En este ejemplo

Horas de estudio y de sueño → ¿aprobó?
Exactitud en prueba: **0.99**.

A6 · Una variable real: glucosa en diabéticos y no diabéticos



Lo que se ajusta

GaussianNB estima, para cada clase, la media μ y desviación σ de la glucosa:

- No diabético: $\mu \approx 108.8$, $\sigma \approx 24.6$
- Diabético: $\mu \approx 138.2$, $\sigma \approx 22.6$

Entre más separadas estén las curvas, más útil es esa variable por sí sola.

A7 · Naive Bayes de texto: multinomial y Bernoulli

```python
from sklearn.feature_extraction.text \
    import CountVectorizer
from sklearn.naive_bayes import (
    MultinomialNB, BernoulliNB)

cv = CountVectorizer()      # cuenta palabras
Xtr = cv.fit_transform(textos_train)

mnb = MultinomialNB().fit(Xtr, ytr)
# o, con presencia/ausencia:
cvb = CountVectorizer(binary=True)
bnb = BernoulliNB().fit(
    cvb.fit_transform(textos_train), ytr)
```

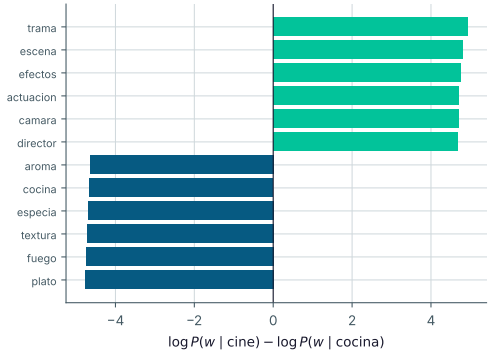
Diferencia clave

Multinomial: usa *cuántas veces* aparece cada palabra. **Bernoulli**: solo si aparece o no, sin importar cuántas veces.

En el corpus de reseñas (cine/cocina)

36 palabras de vocabulario. Exactitud en prueba: Multinomial **1.00**, Bernoulli **1.00**.

A8 · Palabras que más inclinan la decisión



Qué mide la gráfica

$\log P(w \mid \text{cine}) - \log P(w \mid \text{cocina})$ para cada palabra: barras a la derecha $\rightarrow$ típicas de cine; a la izquierda $\rightarrow$ típicas de cocina.

Palabras "comunes" (excelente, aburrido) no inclinan la balanza: aparecen casi igual en ambas clases.

A9 · Un ejemplo, paso a paso

Documento nuevo: `"director escena increible camara"`

Para cada clase se suma el log-prior más el log-verosimilitud de cada palabra (independencia condicional):

$$\log P(\text{cine} \mid x) \propto -12.94 \qquad \log P(\text{cocina} \mid x) \propto -27.00$$

Decisión

Como -12.94 $>$ -27.00, el modelo predice **cine** — razonable: "director", "escena" y "cámara" son vocabulario de cine.

En la práctica se trabaja con logaritmos para evitar multiplicar muchas probabilidades pequeñas (subdesbordamiento numérico).

A10 · Ventajas y límites de Naive Bayes

Ventajas

- Muy **rápido** de entrenar y de predecir; escala bien a miles de variables (ideal para texto).
- Funciona con **pocos datos** por clase.
- Da probabilidades, no solo etiquetas.

Límites

- El supuesto de independencia puede ser **muy falso** y afectar las probabilidades (aunque no siempre la clasificación).
- Variables muy correlacionadas se "cuentan doble".
- GaussianNB asume normalidad; si una variable es muy sesgada, conviene transformarla.

Pausa para pensar (actividad 1, 3 min)

Trabaja de forma individual

1. Un correo contiene la palabra “gratis” 3 veces. ¿Qué variante de Naive Bayes usa esa información y cuál la ignora?
2. Si dos variables de entrada son casi idénticas (muy correlacionadas), ¿qué le pasa a su peso efectivo en la decisión de Naive Bayes?
3. ¿Por qué Naive Bayes es una opción razonable para clasificar miles de correos con decenas de miles de palabras distintas, mientras que un árbol de decisión sería más costoso ahí?

Respuestas: (1) Multinomial usa el conteo (3 veces pesa más que 1); Bernoulli solo ve “presente/ausente”. (2) Se cuenta esencialmente dos veces la misma señal, sesgando la probabilidad (aunque a veces no cambia la clase predicha). (3) Naive Bayes estima cada palabra por separado y es lineal en el número de variables; un árbol tendría que explorar muchísimos posibles cortes entre decenas de miles de palabras.

B1 · De árboles de regresión a árboles de clasificación

Lo que no cambia (sesión 7)

El algoritmo **CART** sigue igual: se buscan cortes "¿$x_i \leq$ umbral?" que dividen el espacio, de forma recursiva (greedy), hasta un criterio de paro.

Lo que cambia

Antes cada hoja predecía un **promedio** (y se medía con MSE). Ahora cada hoja predice la **clase mayoritaria** (o una probabilidad), y se necesita una nueva forma de medir qué tan "buena" es una división: la **impureza**.

B2 · La impureza de Gini

Definición (nodo binario)

$$G = 1 - \sum_{k} p_k^2 = 2\,p\,(1-p)$$

p_k: proporción de la clase k en el nodo. Un nodo **puro** ($p = 0$ o $p = 1$) tiene $G = 0$; el más "revuelto" ($p = 0.5$) tiene $G = 0.5$.

Interpretación

G es, aproximadamente, la probabilidad de clasificar mal si se asignara la clase al azar según las proporciones del nodo.

B3 · Ganancia de información (entropía)

Entropía (nodo binario)

$$H = -\sum_{k} p_k \log_2 p_k$$

Mide la "incertidumbre" en bits; también vale 0 en un nodo puro y es máxima en $p = 0.5$.

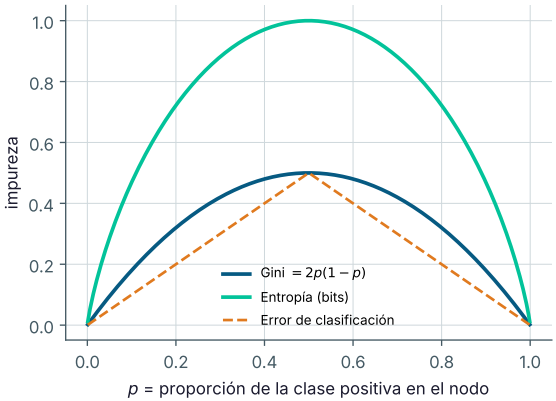
Ganancia de información

Al dividir un nodo en izquierda/derecha, la ganancia es la **reducción** de entropía:

$$\mathrm{IG} = H_{\text{padre}} - \left(\frac{n_I}{n} H_I + \frac{n_D}{n} H_D \right)$$

CART elige el corte que **maximiza** la ganancia (minimiza la entropía ponderada).

B4 · Gini y entropía, en una sola gráfica



Lo importante

Ambas son **cóncavas**, valen 0 en los nodos puros y son máximas en $p = 0.5$: miden esencialmente lo mismo con escalas distintas.

El error de clasificación (línea punteada) también es 0 en los extremos, pero es **lineal** por tramos y menos sensible a los cambios cerca de los extremos; por eso no se usa para elegir cortes.

B5 · Un ejemplo a mano, con los datos de diabetes

Nodo raíz ($n = 2400$)

$p(\text{diabético}) = 0.310 \Rightarrow G_{\text{padre}} = 0.428$

Corte: ¿glucosa ≤ 140?

	n	p_1	G
Izquierda	1906	0.216	0.338
Derecha	494	0.674	0.439

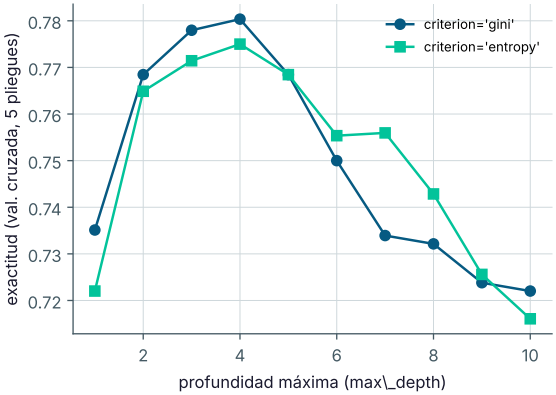
Gini ponderado y ganancia

$$G_{\text{hijos}} = \frac{1906}{2400}(0.338) + \frac{494}{2400}(0.439) = 0.359$$

reducción de Gini $= 0.428 - 0.359 = \mathbf{0.069}$

Con entropía, la misma división da una ganancia de información de **0.108** bits: ambos criterios coinciden en que glucosa ≤ 140 es un corte útil.

B6 · ¿Gini o entropía? Comparación en datos reales



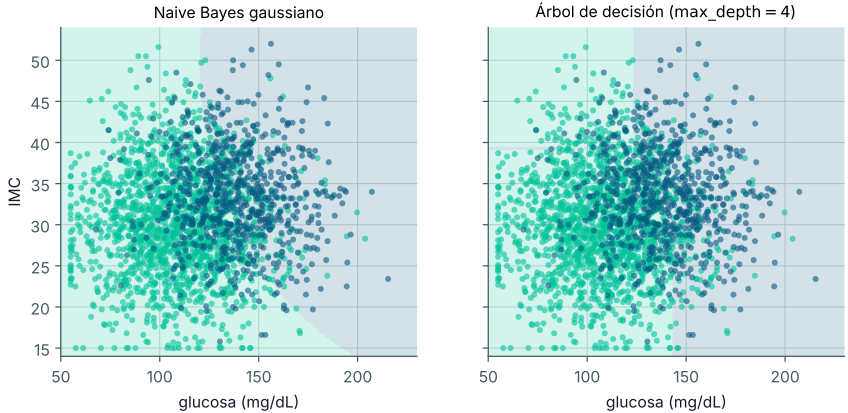
En este conjunto

Mejor `max_depth` con Gini: **4** (exactitud 0.780).
Con entropía: **4** (exactitud 0.775).

La conclusión habitual

Casi siempre dan árboles muy parecidos; Gini es ligeramente más rápido de calcular (sin logaritmos) y es el valor por defecto en scikit-learn.

B7 · Fronteras de decisión: Naive Bayes vs árbol



Mismos datos (glucosa, IMC).

Naive Bayes traza una frontera suave en forma de curva; el árbol traza **rectángulos** (cortes perpendiculares a los ejes).

Exactitud en prueba: Naive Bayes 0.77, árbol 0.76.

B8 · En scikit-learn: DecisionTreeClassifier

```python
from sklearn.tree import (
    DecisionTreeClassifier)
from sklearn.model_selection import (
    GridSearchCV)

arbol = DecisionTreeClassifier(
    criterion="gini",    # o "entropy"
    random_state=2026)

malla = {"max_depth": range(1, 11)}
gs = GridSearchCV(arbol, malla, cv=5,
    scoring="accuracy")
gs.fit(Xtr, ytr)
```

Puntos clave

- `predict_proba` da la proporción de cada clase en la hoja: necesaria para la curva ROC.
- Igual que en regresión: podar con `max_depth` o `min_samples_leaf` evita memorizar.

Pausa para pensar (actividad 2, 3 min)

Trabaja de forma individual

1. Un nodo tiene 90 casos de una clase y 10 de la otra. ¿Su Gini es alto o bajo? ¿Y su entropía?
2. Si un corte deja un hijo con Gini $= 0$ y el otro con Gini $= 0.5$, ¿fue un buen corte? ¿Qué más hay que considerar?
3. ¿Por qué un árbol de decisión, a diferencia de Naive Bayes, **no** necesita el supuesto de independencia entre variables?

Respuestas: (1) Ambos son bajos ($G = 0.18$, $H \approx 0.47$ bits): el nodo ya está bastante puro. (2) Depende del **tamaño** de cada hijo: si el hijo puro tiene pocos casos y el impuro tiene casi todos, la reducción ponderada de impureza es pequeña. (3) Cada corte usa **una sola** variable a la vez y las interacciones emergen de cortes sucesivos, no de un supuesto sobre la distribución conjunta.

C1 · Repaso rápido: la matriz de confusión

De la sesión 8

	pred. 1	pred. 0
real 1	VP=125	FN=98
real 0	FP=72	VN=425

Árbol final (profundidad 4) en el conjunto de prueba de diabetes ($n = 720$): exactitud 0.76, precisión 0.63, recall 0.56, F_1 =0.60.

La pregunta de hoy

Esta matriz corresponde a **un solo umbral** ("clase 1 si $P(1 \mid x) > 0.5$"). ¿Qué pasa si movemos ese umbral?

C2 · De probabilidades a clases: el umbral de decisión

Casi todo clasificador da una probabilidad

`predict_proba` (árboles, Naive Bayes, regresión logística) da $P(1 \mid x) \in [0, 1]$. `predict` solo aplica un umbral, casi siempre 0.5.

La idea clave

El umbral **no tiene que ser** 0.5. Bajarlo detecta más positivos (más recall) a costa de más falsas alarmas (menos precisión), y viceversa.

C3 · Dos tasas, para describir cada umbral

TPR (sensibilidad / recall)

$$\mathrm{TPR} = \frac{\mathrm{VP}}{\mathrm{VP} + \mathrm{FN}}$$

De los positivos reales, cuántos se detectan. (Ya la conocíamos como *recall*.)

FPR

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{VN}}$$

De los negativos reales, cuántos se marcan por error como positivos.

La curva ROC grafica TPR contra FPR, para todos los umbrales posibles

Un buen modelo logra TPR alto con FPR bajo; uno al azar tiene TPR$\approx$FPR en cualquier umbral.

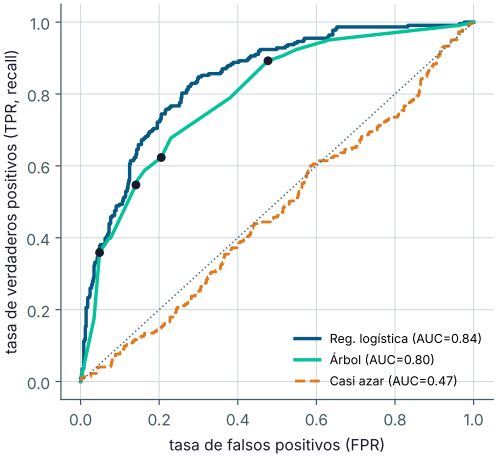
C4 · Construyendo la curva, un umbral a la vez

Tres umbrales del árbol, en el conjunto de prueba

Umbral	FPR	TPR	Lectura
alto (≈ 0.71)	0.048	0.359	muy pocas falsas alarmas, detecta poco
medio (≈ 0.40)	0.205	0.623	un punto intermedio
bajo (≈ 0.10)	0.634	0.951	detecta casi todo, muchas falsas alarmas

Cada umbral da un punto (FPR, TPR); unir **todos** los puntos posibles (de umbral 1 a umbral 0) traza la curva ROC completa.

C5 · La curva ROC completa



Cómo leerla

La diagonal punteada es un clasificador **al azar**. Entre más se acerque la curva a la esquina **superior izquierda** (FPR=0, TPR=1), mejor es el modelo.

Los puntos marcados corresponden a los umbrales de la tabla anterior.

C6 · El área bajo la curva (AUC)

Definición

AUC = área bajo la curva ROC, entre 0 y 1. Equivale a: la probabilidad de que el modelo le dé **mayor** puntaje a un positivo elegido al azar que a un negativo elegido al azar.

Valores de referencia

AUC$= 0.5$: al azar. AUC$= 1.0$: separación perfecta. En nuestros modelos: regresión logística 0.84, árbol 0.80, modelo casi al azar 0.47.

C7 · AUC vs exactitud: ¿por qué AUC es más robusto?

El problema de la exactitud

Con clases desbalanceadas, la exactitud depende **mucho** del umbral elegido (0.5 rara vez es el mejor) y de qué tan común es la clase positiva.

Por qué el AUC ayuda

El AUC resume el desempeño en **todos** los umbrales a la vez; no depende de dónde se ponga el punto de corte, ni de la proporción de clases.

Cuándo preferir cada una

Exactitud/F1: cuando ya se fijó un umbral de uso real (p. ej. un sistema en producción). AUC: para **comparar modelos** entre sí, antes de decidir el umbral final.

C8 · En scikit-learn: `roc_curve` y `roc_auc_score`

```python
from sklearn.metrics import (
    roc_curve, roc_auc_score)

proba = modelo.predict_proba(Xte)[:, 1]
fpr, tpr, umbrales = roc_curve(
    yte, proba)
auc = roc_auc_score(yte, proba)
```

Puntos clave

- `roc_curve` necesita **probabilidades**, no la clase predicha.
- `RocCurveDisplay.from_estimator(...)` dibuja la curva en una línea.
- Solo aplica a problemas **binarios** (hay variantes para multiclase, *one-vs-rest*).

Pausa para pensar (actividad 3, 3 min)

Trabaja de forma individual

1. Dos modelos tienen la misma exactitud, pero uno tiene AUC$= 0.90$ y el otro AUC$= 0.65$. ¿Cuál preferirías y por qué?
2. Si bajas el umbral de decisión de 0.5 a 0.2, ¿la TPR sube o baja? ¿Y la FPR?
3. ¿Tiene sentido calcular el AUC de un modelo que solo entrega `predict` (clases, sin probabilidades)? ¿Por qué?

Respuestas: (1) El de AUC$= 0.90$: separa mucho mejor las clases en general, aunque con el umbral actual su exactitud coincida por casualidad. (2) Ambas suben: un umbral más bajo clasifica más casos como positivos, detectando más verdaderos positivos pero también más falsos positivos. (3) No de forma completa: sin probabilidades solo se tiene un punto (FPR, TPR), no la curva completa; se necesitaría un *score* continuo.

Resumen de la sesión

Ideas clave

- **Naive Bayes**: Bayes + independencia condicional; gaussiano (continuas), multinomial y Bernoulli (texto).
- **Árboles de clasificación**: mismo CART de siempre, ahora con **Gini** o **entropía** para elegir cortes.
- **ROC/AUC**: resumen del desempeño en **todos** los umbrales, no solo en uno.

Para llevarse

- Naive Bayes es rápido y funciona muy bien en texto de alta dimensión; los árboles capturan interacciones y no linealidades sin supuestos distribucionales.
- Gini $\approx$ entropía en la práctica; Gini es el valor por defecto.
- AUC compara modelos; exactitud/F1 evalúan un umbral ya elegido.

Siguiente sesión

Módulo 3: máquinas de vectores de soporte (SVM) para clasificación (sesión 10).

Actividad de laboratorio 1 (individual)

Clasificación de texto con Naive Bayes

Cuaderno de Jupyter con reseñas breves de dos temas (cine / cocina). Trabaja **de forma individual**.

1. Explora el corpus: ¿cuántos documentos por clase? ¿cuántas palabras distintas (vocabulario)?
2. Vectoriza el texto con `CountVectorizer` y entrena un `MultinomialNB`; evalúa con `classification_report` y matriz de confusión.
3. Entrena un `BernoulliNB` con `CountVectorizer(binary=True)` sobre el mismo corpus y compara su exactitud con el multinomial.
4. Identifica las 5 palabras más "típicas" de cada clase usando `feature_log_prob_`.
5. Clasifica tres documentos nuevos que tú mismo escribas y verifica si el resultado es razonable.
6. Interpreta: ¿en qué casos esperarías que Bernoulli y Multinomial dieran resultados muy distintos?

Entregable: el cuaderno con tus respuestas y los tres documentos de prueba que escribiste.

Actividad de laboratorio 2 (individual)

Diagnóstico de diabetes con árboles de decisión

Cuaderno de Jupyter con un conjunto sintético de diagnóstico de diabetes. Trabaja **de forma individual**.

1. Explora los datos: ¿qué proporción de casos es diabético? ¿están desbalanceados?
2. Divide en entrenamiento/prueba (estratificado) y entrena un `DecisionTreeClassifier` con `criterion="gini"`; elige `max_depth` con validación cruzada.
3. Repite con `criterion="entropy"` y compara ambos árboles: ¿cambia mucho el resultado?
4. Calcula la matriz de confusión, precisión, recall y F1 del mejor árbol en el conjunto de prueba.
5. Obtén las probabilidades con `predict_proba`, traza la curva ROC y calcula el AUC.
6. Compara tu árbol contra un `GaussianNB` entrenado con las mismas variables: ¿cuál tiene mejor AUC?

Entregable: el cuaderno con la curva ROC, el AUC de ambos modelos y una conclusión breve sobre cuál recomendarías y por qué.



¡Pasemos a las actividades: texto con Naive Bayes y diabetes con árboles!